COURSE ASSIGNMENT CO1 — SOLUTION & TECHNICAL REPORT

Grama Jyothi Rural Electricity Co-operative

Domain: Embedded C & Data Systems | **Target Scale:** District (800,000 houses) | **Core Focus:** Precision Billing & Architecture

TASK 1 — PROBLEM ANALYSIS & DATA IDENTIFICATION

The Grama Jyothi billing system requires high numerical reliability across scaling transitions (4,000 to 800,000 households). Below is the structured taxonomy of data items:

- **Inputs:** `household_number` (consumer ID), `previous_reading` (16-bit register), `current_reading` (16-bit register), `category` (tariff bracket).
- **Outputs:** `units` (computed consumption), `energy_charge` (slab cost), `fixed_charge` (standing fee), `total` (final bill).
- **Constants:** Slab boundaries (100, 200 units), rates (Rs 0.00, Rs 2.25, Rs 4.50), and fixed charge (Rs 50.00).
- **Core Invariants:** 1) Bills are non-negative ($Total \ge 0$). 2) $Total = FixedCharge + EnergyCharge$. 3) Rollover arithmetic must satisfy modulo 2^{16} wrapping.

Alternative Concept & Complexity Analysis (String vs Integer Representation):

If readings were stored as strings (e.g., `"01530"`), conversion to numerical values for math requires $O(d)$ time per reading (d = digits). Integer storage requires $O(1)$ time for native arithmetic. Space-wise, a 5-digit ASCII string uses 5 bytes (or 6 with null terminator), whereas a 16-bit `unsigned short` integer requires exactly 2 bytes. Integer representation provides 60% memory savings and instant arithmetic execution.

TASK 2 — CHOOSE THE DATA TYPES & METER ROLLOVER

Data Item	C Data Type	Size (Bytes)	Range (Standard Limits)	Justification
Meter Reading	`unsigned short`	2	0 to 65,535	Matches physical 16-bit hardware counters.
Consumption	`unsigned int`	4	0 to 4,294,967,295	Prevents overflow on large aggregated usages.
Financial Amounts	`long` / `long long`	4 / 8	$\pm 2.14 \times 10^9$ / $\pm 9.22 \times 10^{18}$	Stores exact **paise** integers; avoids float drift.

Why Float Rupees Fail: Floating-point numbers use binary fractions (2^{-n}), which cannot represent decimal fractions like 0.10 or 0.25 precisely. Accumulating 4,000 float bills introduces compound rounding errors, causing off-by-paise discrepancies. Storing currency as integral paise eliminates all floating-point inaccuracy.

Code Implementation: Task 2 Solution

```
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current) {
    // Unsigned 16-bit subtraction handles register roll-over automatically:
    // (120 - 65400) mod 65536 = 256
    return (unsigned int)((meter_t)(current - previous));
}

int main(void) {
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n", sizeof(meter_t),
(unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120)); /* rolled over */
    return 0;
}
```

TASK 3 — DEFINE CONSTANTS & TARIFF CALCULATION

Centralizing constants ensures single-point modifications for tariff revisions. `#define` macros avoid memory allocation and overhead, whereas `const` variables provide scope control and explicit type checking.

Code Implementation: Task 3 Solution

```
#include <stdio.h>

#define SLAB1_UNITS         100
#define SLAB2_UNITS         200
#define RATE1_PAISE         0L      /* First 100 units free */
#define RATE2_PAISE         225L    /* Rs 2.25 per unit */
#define RATE3_PAISE         450L    /* Rs 4.50 per unit */
#define FIXED_CHARGE_PAISE  5000L   /* Rs 50.00 fixed charge */

long compute_bill(int units) {
    long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS) {
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
        paise += (long)(SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE;
    } else if (units > SLAB1_UNITS) {
        paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
    }
    /* units <= SLAB1_UNITS incur 0 energy charge */

    return paise;
}

int main(void) {
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof(tests) / sizeof(tests[0]);
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
    }
    return 0;
}
```

TASK 4 — SAFE MACRO ENGINEERING

Macros must be fully parenthesized to ensure evaluation order protection. Without outer and argument parentheses, expressions like `SQUARE(a+1)` expand incorrectly to `a+1*a+1`.

- **Unsafe Macro:** `#define SQUARE(x) x * x` $\rightarrow$ `SQUARE(2+3)` evaluates as $2 + 3 \times 2 + 3 = 11$ (Incorrect).
- **Safe Macro:** `#define SQUARE(x) ((x) * (x))` $\rightarrow$ `((2+3) * (2+3))` = 25 (Correct).
- **Side-Effect Pitfall:** `#define MAX(a,b) ((a) > (b) ? (a) : (b))` evaluates `MAX(i++, j)` twice on the larger operand, mutating `i` twice. Inline functions should be used when side-effect safety is mandatory.

TASK 5 & 6 — HEADER ARCHITECTURE & INPUT PARSING

Modular architecture uses header guards (`#ifndef BILLING_H ... #endif`) to prevent duplicate definitions. Conditional compilation via `#ifdef DEBUG` allows diagnostic telemetry to be compiled out of production binaries without runtime penalty.

Input Parsing Safety (Task 6):

Always validate `scanf()` return values to catch malformed inputs:

```
if (scanf("%hu %hu", &prev, &curr) != 2) {
    fprintf(stderr, "Error: Invalid non-numeric input detected.\n");
    return 1;
}
```

TASK 7 & 8 — TYPE CONVERSIONS & BUILD STAGES

- **Implicit Conversions:** Mixing `int` and `long` automatically promotes variables to higher rank. However, integer division like `5 / 2` truncates to 2. Always cast explicitly: `(double)a / b`.
- **Compilation Stages:**
  1. `gcc -E` (Preprocessing): Expands headers and macros; removes comments.
  2. `gcc -S` (Compilation): Translates code to assembly language.
  3. `gcc -c` (Assembly): Generates binary object file (`.o`).
  4. `gcc -o` (Linking): Combines object code with C runtime libraries.

TASK 9 — VERIFICATION & EMPIRICAL TEST RESULTS

Test Case	Previous	Current	Units	Expected Bill	Actual Output	Status
Zero Usage	1000	1000	0	Rs 50.00	Rs 50.00	PASS
Slab 1 Limit	1000	1100	100	Rs 50.00	Rs 50.00	PASS
Slab 2 Midpoint	1000	1150	150	Rs 162.50	Rs 162.50	PASS
Slab 2 Upper Boundary	1000	1200	200	Rs 275.00	Rs 275.00	PASS
Slab 3 High Usage	1000	1350	350	Rs 950.00	Rs 950.00	PASS
Register Rollover	65400	120	256	Rs 527.00	Rs 527.00	PASS

TASK 10 & 11 — SCALE COMPARISON & ARCHITECTURE RECOMMENDATIONS

District-Scale Scalability Analysis (800,000 Households):

At 800,000 households, standard 32-bit `int` cumulative paise counters overflow at Rs 2.14 Crore. Accumulators at district level must use `long long` (64-bit integer), which supports totals up to Rs 92,233,720,368,547,758.07 before overflow.

- **Final Recommendation:** Store all monetary transactions in integer paise using `long long`. Use `unsigned short` for hardware registers and `unsigned int` for consumption units.
- **Tariff Maintenance:** Move tariff constants out of `#define` preprocessor macros into an external binary/JSON runtime config block to support zero-recompile policy changes.

TASK 12 — SMART METER ARCHITECTURE RE-EVALUATION

> **Design Challenge Transition:** Smart meters operating at 15-minute intervals generate 35,040 readings per household/year (28.03×10^9 annual readings district-wide), introducing Time-of-Day (ToD) peak pricing.

A. Impact Analysis

- **Data Volume:** 15-minute reporting increases dataset size by factor $2,920\times$. 16-bit counters are no longer sufficient for aggregated yearly energy totals; counters must upgrade to 64-bit `unsigned long long` (watt-hours).
- **Time-of-Day Tariff:** Usage between 6 PM – 10 PM incurs a +20% tariff multiplier. Consumption must be logged in hourly buckets rather than a single scalar scalar monthly counter.
- **Dynamic Configuration:** Compile-time `#define` macros must be discarded in favor of runtime configuration tables loaded from disk to enable instant board updates without re-compilation.

B. Redesigned Smart Meter Data Structure (C)

```
typedef struct {
    unsigned long long total_wh;       /* Watt-hours total */
    unsigned int peak_wh;              /* Peak period (6 PM - 10 PM) */
    unsigned int off_peak_wh;          /* Off-peak period */
} SmartReading;

typedef struct {
    long base_rate_paise_per_kwh;
    int peak_multiplier_percent;       /* e.g., 120 for 20% surcharge */
    long fixed_charge_paise;
} DynamicTariffConfig;
```